



به نام خدا
دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر



پروژه ی درس درستی سنجی مدل های HDL

مهسا ابرغانی اقدام - ۸۱۰۱۰۳۰۵۳

مرحله ی دوم: پیاده سازی واحد Stimuli Generator

- از این سناریو ها corner case ها رو مشخص کنید:

۱. عملیات خواندن و نوشتن همزمان: اگر سیگنال های نوشتن و خواندن به صورت همزمان فعال شوند، سیستم ممکن است در تصمیم گیری برای پردازش عملیات دچار اختلال شود. این مورد می تواند به رفتار غیرمنتظره ای در خروجی منجر شود.
۲. دسترسی به مرزهای بانک ها: نوشتن و خواندن داده ها در آدرس هایی که به مرز بانک ها نزدیک هستند (برای مثال، آخرین آدرس بانک ۰ و اولین آدرس بانک ۱) ممکن است باعث شود سیستم در تغییر بانک دچار اختلال شود
۳. دسترسی به حداکثر تعداد آدرس ها: تست سیستم با نوشتن و خواندن از بیشترین تعداد آدرس های موجود (مثلاً استفاده از تمامی آدرس های موجود در بانک ۰) می تواند ظرفیت و تحمل سیستم را بررسی کند. چراکه ممکن است که سیستم نتواند این حجم از داده را مدیریت کند.
۴. خواندن یا نوشتن در مقادیر مرزی: نوشتن و خواندن داده هایی با مقادیر مرزی می تواند نشان دهنده مشکلاتی مانند سرریز بافر یا تغییرات ناخواسته در بیت های داده باشد. این حالت به ویژه زمانی مهم است که داده های مرزی با سرعت بالا خوانده و نوشته شوند.

۵. عملیات سریع NOP، بعد از خواندن یا نوشتن: اگر بلافاصله پس از عملیات نوشتن یا خواندن، چندین NOP سریع ارسال شود، سیستم باید بتواند به درستی بین حالت‌ها سوئیچ کند. این مورد برای بررسی پایداری سیستم در شرایط با تغییر سریع است.
۶. ریست کردن سیستم در حین عملیات فعال: اگر سیستم در حالی که یک عملیات خواندن یا نوشتن در حال اجراست، ریست شود، باید به طور صحیح به حالت اولیه برگردد و از بروز مشکلاتی مانند نگهداری داده‌های ناتمام جلوگیری شود.
۷. تاخیرهای طولانی بین عملیات متوالی: ایجاد تأخیر طولانی بین دو عملیات متوالی می‌تواند بررسی کند که آیا سیستم در نگهداری داده‌ها و وضعیت‌های داخلی خود پایداری لازم را دارد یا خیر.
-

- در این مرحله باید برای سناریوها و زیر سناریوهای مشخص شده یک واحد Stimuli Generator پیاده سازی نمایید. این واحد توانایی آماده سازی سناریو برای سیستم را داشته باشد. خروجی مرحله فایل `determenstic_stimuli_genrator.sv` است که ورودیها مربوط به سناریوها را آماده میکند و به سیستم ما اعمال می کند.

سناریو ها و اجرای آن ها:

سناریوی ۱: عملیات ساده نوشتن

این سناریو برای بررسی صحت عملکرد نوشتن در SDRAM طراحی شده است. هدف این است که اطمینان حاصل شود SDRAM می‌تواند درخواست‌های نوشتن را به درستی پردازش کند و داده‌ها را در مکان مشخص شده ذخیره نماید یا خیر.

```
// Scenario 1: Simple Write Operation
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 1;
tb_HWDATA = 32'hDEADBEEF;
tb_HADDR = 32'h80000000;
#50;
#1;
$fwrite(file, "Scenario 1: Time: %0t | tb_HREADY: %b | tb_HRDATA: %h\n", $time, tb_HREADY, tb_HRDATA);
```

خروجی:

Scenario 1: Time: 176 | tb_HREADY: 0 | tb_HRDATA: xxxxxxxx

در این مرحله، عملیات نوشتن انجام شده است، اما هنوز داده‌ای برای خواندن وجود ندارد. مقدار `tb\_HRDATA` برابر با `XXXXXXXX` نشان‌دهنده این است که داده‌ای خوانده نشده است و حافظه در حالت نوشتن بوده است.

سناریوی ۲: عملیات ساده خواندن

این سناریو عملکرد خواندن را بررسی می‌کند. هدف این است که مشخص شود آیا SDRAM می‌تواند داده‌های ذخیره‌شده را به درستی بازیابی کرده و در اختیار سیستم قرار دهد.

```
// Scenario 2: Simple Read Operation
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 0;
tb_HADDR = 32'h80000000;
#50;
#1;
$fwrite(file, "Scenario 2: Time: %0t | tb_HREADY: %b | tb_HRDATA: %h\n", $time, tb_HREADY, tb_HRDATA);
#100;
```

خروجی:

Scenario 2: Time: 236 | tb_HREADY: 1 | tb_HRDATA: deadbeef

در این سناریو، عملیات خواندن انجام شده و مقدار ذخیره‌شده (`deadbeef`) با موفقیت از آدرس مربوطه بازیابی شده است. مقدار `tb\_HREADY: 1` نشان می‌دهد که حافظه آماده پاسخدهی است.

سناریوی ۳: نوشتن، ریست کردن، و خواندن

این سناریو رفتار SDRAM را در شرایطی که سیستم ریست می‌شود بررسی می‌کند. هدف این است که مشخص شود ریست کردن، عملکرد SDRAM یا داده‌های ذخیره‌شده را تحت تأثیر قرار نمی‌دهد و سیستم پس از ریست به درستی عمل می‌کند.

```
// Scenario 3: Write, Reset, and Read Operation
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 1;
tb_HWDATA = 32'h55555555;
tb_HADDR = 32'h89444444;
#20;
tb_HRESET = 1;
#10;
tb_HRESET = 0;
#60;
tb_HSEL = 1;
tb_HWRITE = 0;
tb_HADDR = 32'h89444444;
#1;
$fwrite(file, "Scenario 3: Time: %0t | tb_HREADY: %b | tb_HRDATA: %h\n", $time, tb_HREADY, tb_HRDATA);
#100;
```

خروجی:

Scenario 3: Time: 436 | tb_HREADY: 0 | tb_HRDATA: deadbeef

در این سناریو، پس از ریست کردن سیستم، داده‌ها همچنان بدون تغییر باقی مانده‌اند و مقدار قبلی (`deadbeef`) به درستی خوانده شده است. این نتیجه نشان می‌دهد که ریست کردن، داده‌های ذخیره‌شده را تحت تأثیر قرار نداده است.

سناریوی ۴: عملیات چندگانه نوشتن

این سناریو عملکرد SDRAM را در شرایطی که چندین عملیات نوشتن پشت سر هم انجام می‌شود، بررسی می‌کند. هدف این است که پایداری و توانایی مدیریت درخواست‌های متوالی ارزیابی شود.

```
// Scenario 4: Multiple Write Operations
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 1;
tb_HWDATA = 32'hCAFEFABE;
tb_HADDR = 32'h80000004;
#50;
#1;
$fwrite(file, "Scenario 4: Time: %0t | tb_HREADY: %b | tb_HRDATA: %h\n", $time, tb_HREADY, tb_HRDATA);

@(posedge tb_HCLK);
tb_HWDATA = 32'hBADCF0DE;
tb_HADDR = 32'h80000008;
#50;
#1;
$fwrite(file, "Scenario 4: Time: %0t | tb_HREADY: %b | tb_HRDATA: %h\n", $time, tb_HREADY, tb_HRDATA);
```

خروجی:

Scenario 4: Time: 596 | tb_HREADY: 0 | tb_HRDATA: 55555555

Scenario 4: Time: 656 | tb_HREADY: 0 | tb_HRDATA: 55555555

در این سناریو، چندین عملیات نوشتن متوالی انجام شده است و داده‌های قبلی به درستی نوشته و ذخیره شده‌اند. مقدار `tb\_HRDATA` در هر دو زمان یکسان است، که نشان‌دهنده موفقیت در نوشتن و خواندن داده‌ها در عملیات متوالی است.

سناریوی ۵: تست مرز بانک

در این سناریو، رفتار SDRAM هنگام دسترسی به مرزهای بین بانک‌ها بررسی می‌شود. هدف این است که مشخص شود SDRAM می‌تواند آدرس‌دهی بانک‌ها را به درستی مدیریت کند و تداخلی در داده‌ها ایجاد نشود.

```
// Scenario 5: Bank Boundary Test
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 1;
tb_HWDATA = 32'h12345678;
tb_HADDR = 32'h88000000;
#50;
#1;
$fwrite(file, "Scenario 5: Time: %0t | tb_HREADY: %b | tb_HRDATA: %h\n", $time, tb_HREADY, tb_HRDATA);
#100;
```

خروجی:

Scenario 5: Time: 716 | tb_HREADY: 0 | tb_HRDATA: 55555555

این خروجی نشان می‌دهد که هنگام دسترسی به مرز بین بانک‌ها، داده‌ها همچنان صحیح باقی مانده‌اند و SDRAM توانسته است آدرس‌دهی بانک‌ها را به درستی مدیریت کند.

سناریوی ۶: خواندن متوالی پس از نوشتن در بانک‌های مختلف

این سناریو بررسی می‌کند که آیا SDRAM می‌تواند داده‌های نوشته‌شده در بانک‌های مختلف را به درستی مدیریت و بازیابی کند. هدف، اطمینان از صحت عملکرد در شرایطی است که درخواست‌های متوالی برای بانک‌های مختلف وجود دارد.

```
// Scenario 6: Sequential Read After Write in Different Banks
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 0;
tb_HADDR = 32'h88000000;
#60;
#1;
$fwrite(file, "Scenario 6: Time: %0t | tb_HREADY: %b | tb_HRDATA: %h\n", $time, tb_HREADY, tb_HRDATA);
#100;
```

خروجی:

Scenario 6: Time: 886 | tb_HREADY: 0 | tb_HRDATA: 55555555

این نتیجه نشان می‌دهد که داده‌های ذخیره‌شده در بانک‌های مختلف به درستی قابل بازیابی هستند. این سناریو توانایی SDRAM در مدیریت درخواست‌های متوالی برای بانک‌های مختلف را تأیید می‌کند.

سناریوی ۷: ریست حین عملیات

این سناریو رفتار SDRAM را هنگام ریست شدن سیستم در حین انجام عملیات بررسی می‌کند. هدف این است که مطمئن شویم ریست باعث ایجاد ناپایداری یا خرابی در عملکرد SDRAM نمی‌شود.

```
// Scenario 7: Reset During Operation
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 1;
tb_HWDATA = 32'hAABBCCDD;
tb_HADDR = 32'h80000000;
#30;
tb_HRESET = 1;
#20;
tb_HRESET = 0;
#10;
$fwrite(file, "Scenario 7: Time: %0t | tb_HREADY: %b | Reset Triggered\n", $time, tb_HREADY);
#100;
```

خروجی:

Scenario 7: Time: 1055 | tb_HREADY: 0 | Reset Triggered

در این سناریو، ریست شدن سیستم در حین عملیات اتفاق افتاده است. مقدار 'Reset Triggered' نشان می‌دهد که عملیات در حال اجرا به درستی متوقف شده و سیستم دوباره ریست شده است. این نتیجه نشان‌دهنده رفتار پایدار SDRAM تحت شرایط خاص است.

خروجی مدلسیم:

